SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION **COURSE CODE:ITA0510** **Slot A**

COURSE OUTCOME COVERED

CO3: ***Analyze and extract image features using detection and matching techniques for effective image representation and comparison.*** **(BTL 4)**

Assessment Tool Weightage: 40%

QUESTIONS:3 **Total Marks 30** **Duration :60 Minutes Annexure A**

Experiments

Code of Conduct:

I _M.Manoj Kumar(192425114)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **M. Manoj Kumar**

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	

Documentation	**10**	**Clear, well-structured documentation with diagrams, code, and results**	**Organized documentation with necessary details**	**Basic documentation with missing details**	**Poor or incomplete documentation**	

1. Preprocessing Impact Analysis

Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.

Aim

To study the impact of preprocessing techniques such as **noise removal** and **normalization** on image quality and feature detection using OpenCV.

Theory

Image preprocessing is the first step in computer vision. It improves image quality before feature extraction and object detection.

Noise Removal

Noise is the unwanted variation in pixel values caused by sensors or transmission errors. It reduces image quality and makes feature detection difficult. Gaussian Blur is commonly used to smooth the image and reduce noise.

Normalization

Normalization adjusts pixel intensity values to a standard range (0–255). It improves image brightness and contrast, making edges and important features easier to detect.

Advantages

- Removes unwanted noise.
- Improves image quality.
- Enhances contrast.
- Improves edge detection.
- Increases feature detection accuracy.

Software Required

- Python 3.x

- OpenCV (cv2)
- VS Code

Algorithm

1. Read the input image.
2. Apply Gaussian Blur to remove noise.
3. Normalize the image intensity values.
4. Display the original, blurred, and normalized images.
5. Save the processed images.
6. Compare the outputs and analyze the results.

Python Code import cv2 img = cv2.imread("beautiful-natural-

image-1844362_1280.jpg")

Save original cv2.imwrite("original.jpg",

img)

Noise removal blur =

cv2.GaussianBlur(img, (5,5), 0)

cv2.imwrite("noise_removed.jpg", blur)

Normalization norm = cv2.normalize(blur, None, 0, 255,

cv2.NORM_MINMAX) cv2.imwrite("normalized.jpg", norm)

Display all three

cv2.imshow("Original", img)

cv2.imshow("Noise Removed", blur)

```
cv2.imshow("Normalized",     norm)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Input

Output

Output 1 – Original Image

Output 2 – Noise Removed Image

Output 3 – Normalized Image

Analysis

- The original image contains fine details such as trees, grass, mountains, lake, and clouds.
- Gaussian Blur reduces small noise and smooths the image while preserving important structures.
- Normalization improves brightness and contrast, making the image appear clearer.
- The processed images help feature detection algorithms identify edges and key points more accurately.
- Preprocessing increases the reliability of image analysis in computer vision applications.

Conclusion

The experiment demonstrates that preprocessing significantly improves image quality. Gaussian Blur effectively reduces noise, while normalization enhances contrast and visibility. These techniques improve the accuracy and reliability of feature detection, making them essential preprocessing steps in computer vision applications.

Result

The preprocessing techniques (Gaussian Blur and normalization) were successfully applied to the input landscape image. The processed images showed improved image quality, making important features such as trees, mountains, and clouds clearer for further computer vision tasks.

2. Image Representation Study

Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

Aim

To convert a given image into **grayscale** and **binary** representations and analyze their effect on edge detection and feature extraction.

Theory

Image representation is the process of expressing an image in different formats for computer vision applications.

Grayscale Image

A grayscale image contains only shades of gray, with pixel values ranging from **0 (black)** to **255 (white)**. It reduces computational complexity while preserving important image details.

Binary Image

A binary image contains only **two pixel values: 0 (black) and 255 (white)**. It is obtained by applying a threshold to a grayscale image. Binary images are useful for object segmentation and shape detection.

Advantages

- Reduces image complexity.
- Improves processing speed.
- Makes edge detection easier.
- Useful for feature extraction and object recognition.

Software Required

- Python 3.x
- OpenCV (cv2)
- VS Code

Algorithm

1. Read the input image.
2. Convert the image to grayscale.
3. Convert the grayscale image into a binary image using thresholding.
4. Display the original, grayscale, and binary images.
5. Save the generated images.
6. Compare the outputs.

Python Code import cv2 # Read image img =

cv2.imread("beautiful-natural-image-1844362_1280.jpg")

if img is None:

print("Image not found!")

exit()

Convert to Grayscale gray = cv2.cvtColor(img,

cv2.COLOR_BGR2GRAY)

Convert to Binary

_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

Save Images cv2.imwrite("original.jpg",

img) cv2.imwrite("grayscale.jpg", gray)

cv2.imwrite("binary.jpg", binary)

Display Images cv2.imshow("Original

Image", img) cv2.imshow("Grayscale

Image", gray) cv2.imshow("Binary

Image", binary) cv2.waitKey(0)

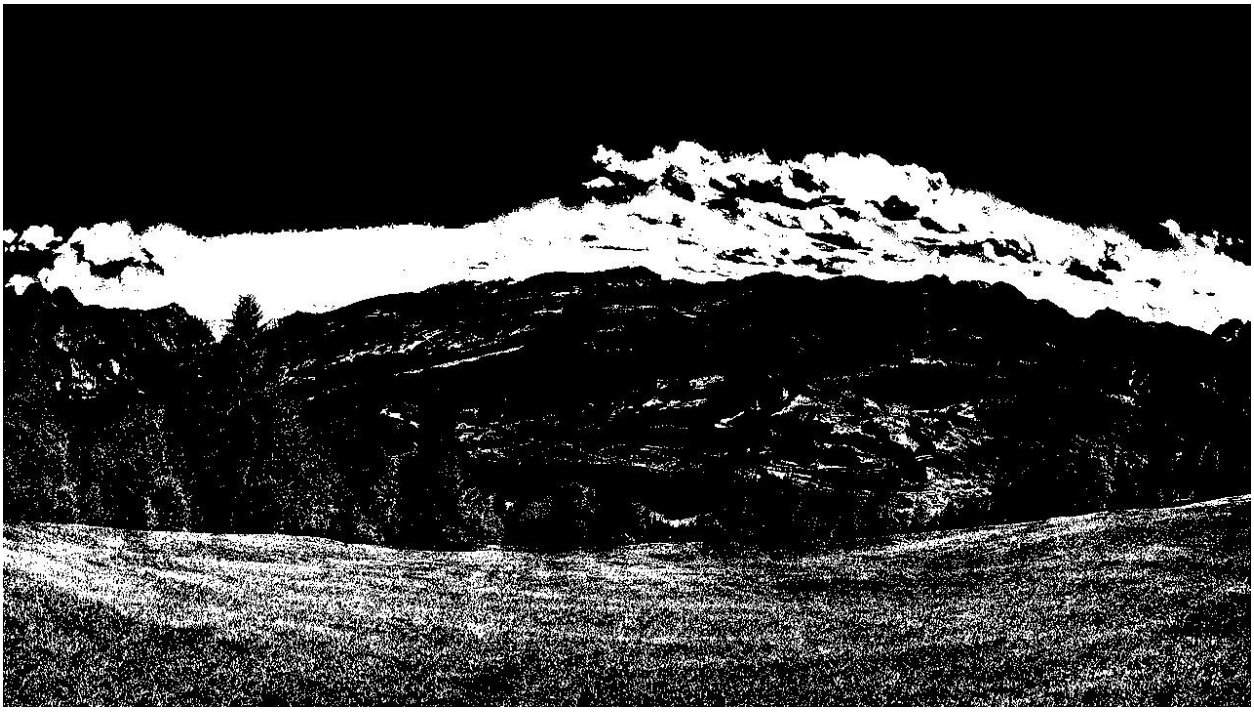
cv2.destroyAllWindows()

Input

Output – 1 – Original Image

Output 2 – Grayscale Image

Output 3 – Binary Image



Analysis

- The original image contains color information and fine details.

- The grayscale image removes color information while preserving intensity variations, making it suitable for edge detection.
- The binary image separates the foreground and background using thresholding, simplifying object detection.
- Grayscale representation is more effective for feature extraction, whereas binary representation is useful for segmentation and shape analysis.

Conclusion

The experiment successfully converted the input image into grayscale and binary representations. The grayscale image reduced computational complexity while preserving important features, whereas the binary image simplified the image for segmentation and object detection. These representations are widely used in computer vision applications.

Result

The input landscape image was successfully converted into grayscale and binary representations. The grayscale image preserved intensity information, while the binary image clearly separated pixels into black and white regions, demonstrating different image representation techniques.

3. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

Aim

To compare **Sobel** and **Canny** edge detection techniques on the same input image and analyze their effectiveness in detecting edges.

Theory

Edge detection is a technique used in computer vision to identify object boundaries by detecting sudden changes in pixel intensity.

Sobel Edge Detection

The Sobel operator calculates the intensity gradient in the horizontal and vertical directions. It highlights edges but is sensitive to noise and may produce thicker edges.

Canny Edge Detection

The Canny algorithm is a multi-stage edge detection technique that includes noise reduction, gradient calculation, non-maximum suppression, and double thresholding. It produces thin, accurate, and continuous edges.

Advantages

- Detects object boundaries.
- Helps in image segmentation.
- Useful for feature extraction and object recognition.
- Improves image analysis.

Software Required

- Python 3.x
- OpenCV (cv2)
- VS Code

Algorithm

1. Read the input image.
2. Convert the image to grayscale.
3. Apply Sobel edge detection.
4. Apply Canny edge detection.
5. Display and save the output images.
6. Compare the results.

Python Code import cv2 # Read image img =

cv2.imread("beautiful-natural-image-1844362_1280.jpg")

if img is None:

print("Image not found!")

exit()

Convert to grayscale gray = cv2.cvtColor(img,

cv2.COLOR_BGR2GRAY)

Sobel Edge Detection sobel = cv2.Sobel(gray,

cv2.CV_64F, 1, 1, ksize=3) sobel =

cv2.convertScaleAbs(sobel)

Canny Edge Detection canny =

cv2.Canny(gray, 100, 200)

Save Images cv2.imwrite("original.jpg",

img) cv2.imwrite("sobel.jpg", sobel)

cv2.imwrite("canny.jpg", canny) #

Display Images cv2.imshow("Original

Image", img) cv2.imshow("Sobel Edge

Detection", sobel) cv2.imshow("Canny

Edge Detection", canny) cv2.waitKey(0)

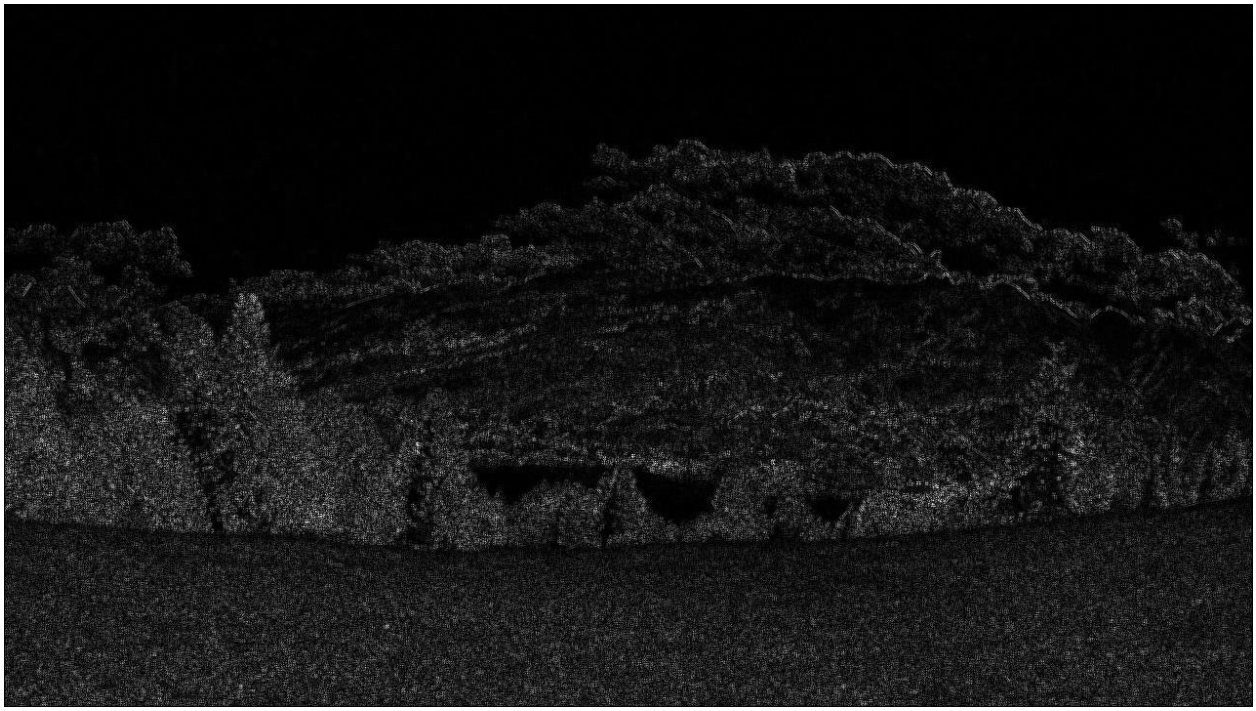
cv2.destroyAllWindows()

Input

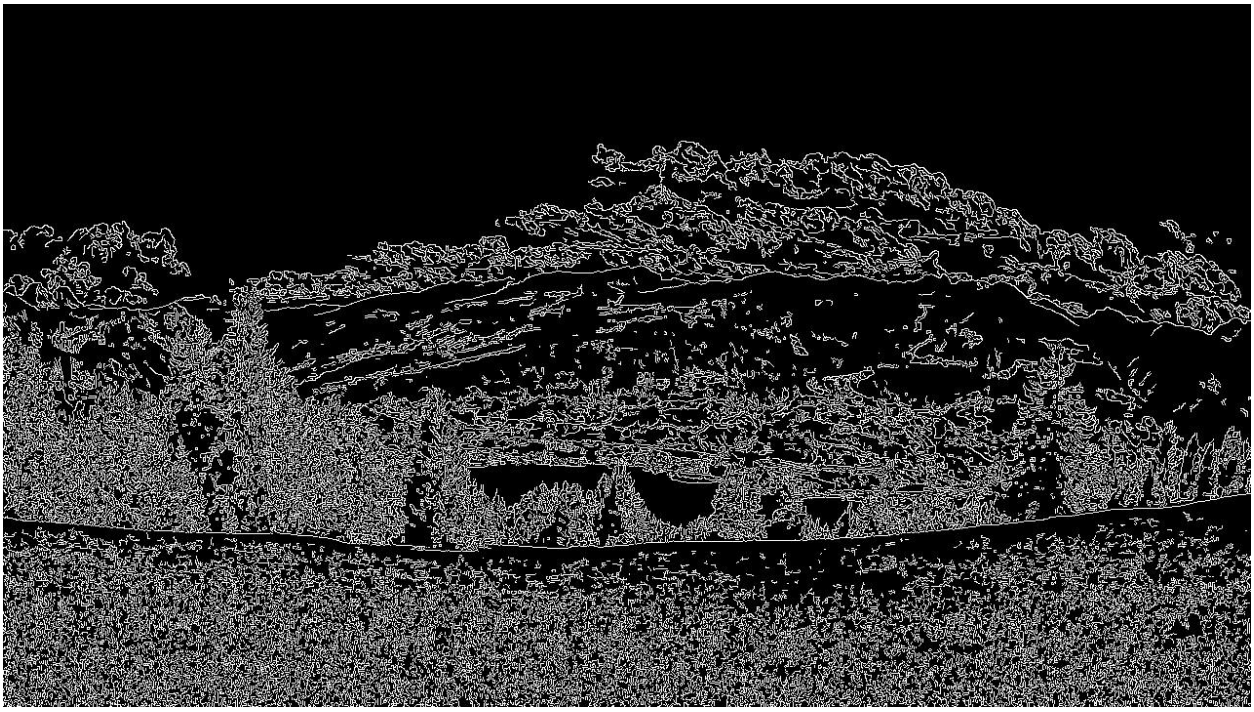
Output

Output 1 – Original Image

Output 2 – Sobel Edge Detection



Output 3 – Canny Edge Detection



Analysis

- The Sobel method detects horizontal and vertical intensity changes, producing thicker edges and detecting many image details.
- The Canny method removes noise before edge detection and produces thinner, clearer, and more continuous edges.
- In the landscape image, Canny clearly highlights the outlines of trees, mountains, lake, and clouds, while Sobel detects more texture but also includes extra edge responses.
- Canny provides better edge localization and is more suitable for feature extraction.

Conclusion

The experiment successfully compared Sobel and Canny edge detection techniques. Sobel effectively detects image gradients but may produce thicker edges. Canny generates sharper and more accurate edges by reducing noise before edge detection. Therefore, Canny is generally more effective for computer vision applications requiring precise edge detection.

Result

The Sobel and Canny edge detection techniques were successfully applied to the input image. Sobel detected image gradients with thicker edges, whereas Canny produced thin, well-defined, and accurate edges. The comparison shows that Canny provides better edge detection for most computer vision applications.